

Jour 3 à 5 - Mémoire et gestion des erreurs des

Langages compilés

Objectifs du module



01

Introduction au C

Appréhender les langages compilés par la pratique du C



02

Comprendre la gestion mémoire

Utiliser et optimiser la mémoire allouée aux programmes compilés



03

Pratiquer le C

Améliorer sa logique de programmation et sa gestion des erreurs



04

Partager

Présenter un projet fonctionnel en C

AGENDA



18/05

Introduction

20/05

Pointeurs et éléments techniques

08/06

Mémoire et gestion des erreurs 🐛▶□

09/06

Pratique et projet ?

15/06

Evaluation projet ▶

Evaluations

Restitution de connaissances

08/06

Valider que vous avez compris les notions vu
ensemble lors des premiers cours.

Présentation orale d'un projet en C

15/06

Montrer et prouver vos compétences et votre
prise en mains sur les langages compilés

Sommaire

- 01** Rappels de la séance précédente
- 02** Gestion de la mémoire en C
- 03** Gestion des erreurs
- 04** Projet fil rouge



Objectifs du cours

01



Comprendre les notions fondamentales de la gestion mémoire

02



Identifier et résoudre les erreurs de compilation

03



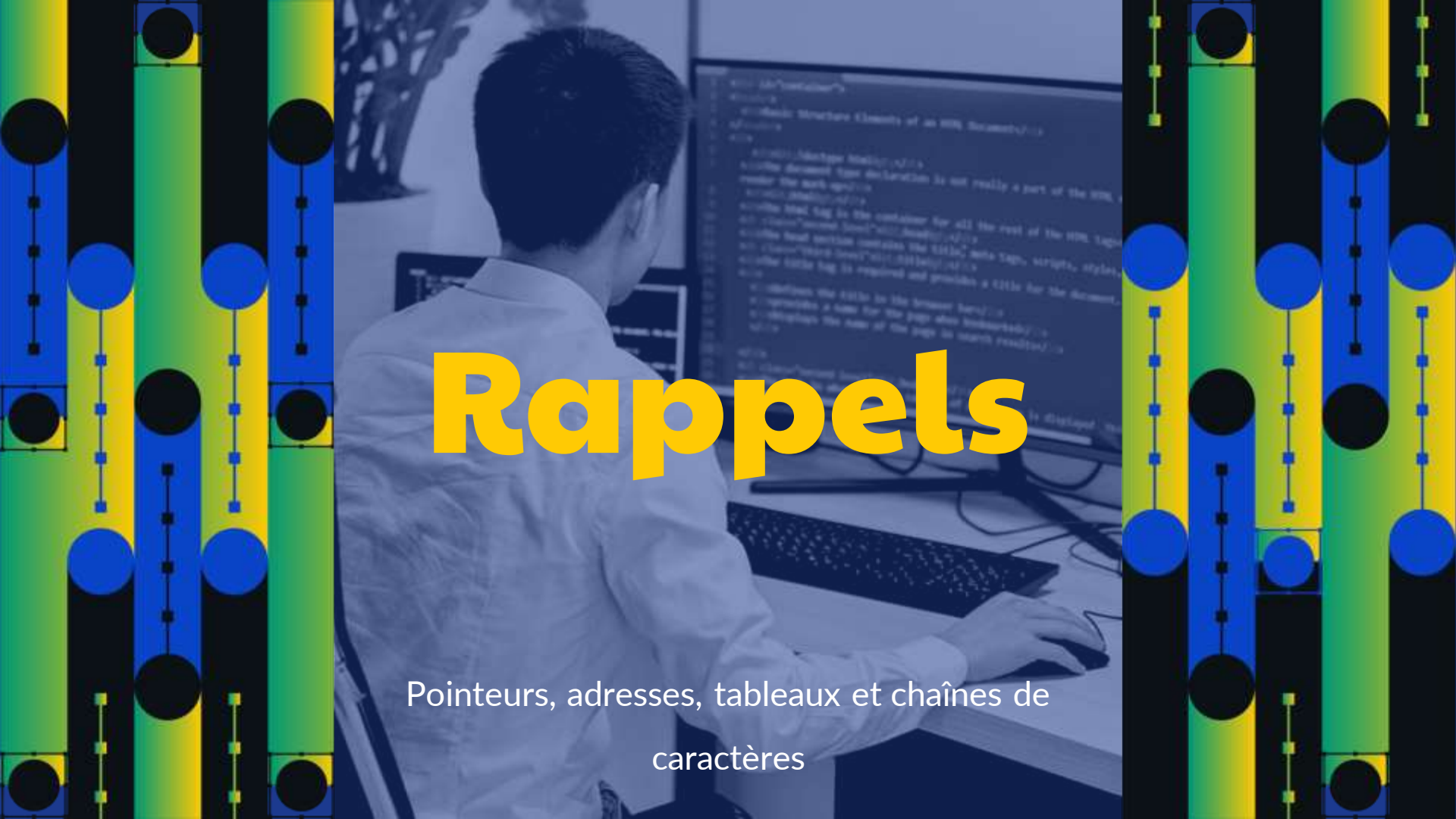
Connaître la différence entre la compilation et le runtime

04



Programmer des applications simples en utilisant des langages compilés



The image features a person from behind, sitting at a desk and looking at a computer monitor. The monitor displays some code. The entire image is covered with a semi-transparent blue filter. On the left and right sides, there are decorative vertical borders made of alternating yellow and green bars, with black circles and small squares placed on them. A large yellow title is centered over the image.

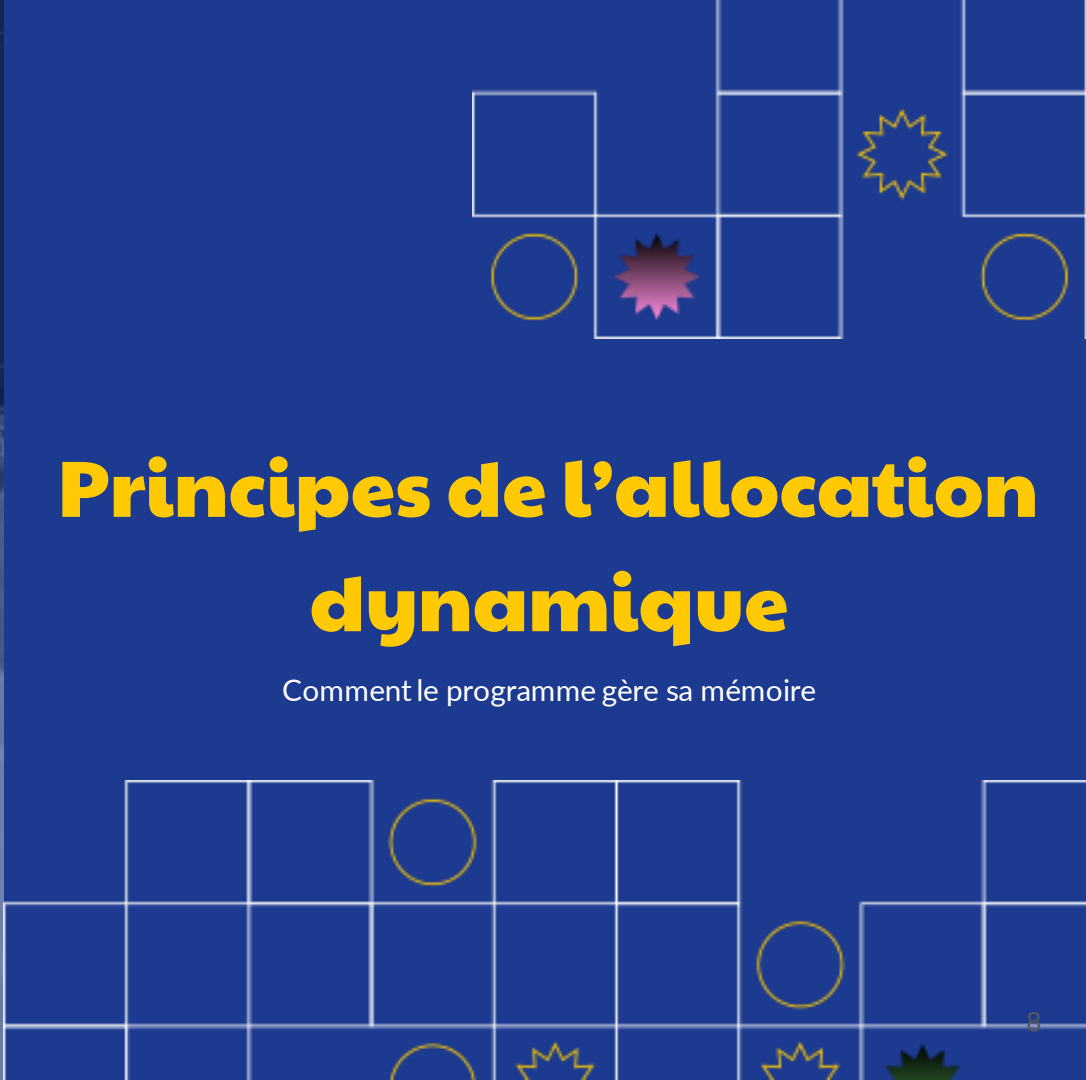
Rappels

Pointeurs, adresses, tableaux et chaînes de
caractères



Principes de l'allocation dynamique

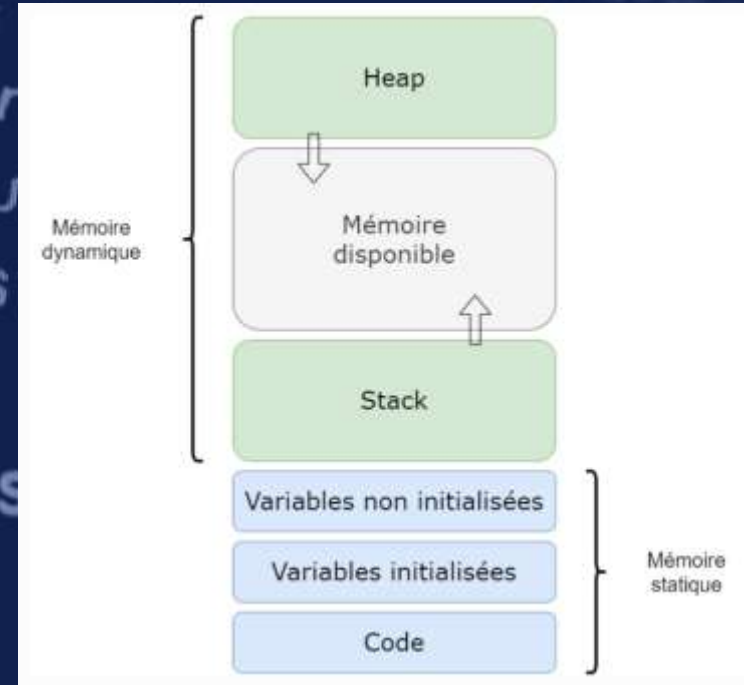
Comment le programme gère sa mémoire



Mémoire - Organisation

La mémoire est divisée en plusieurs segments :

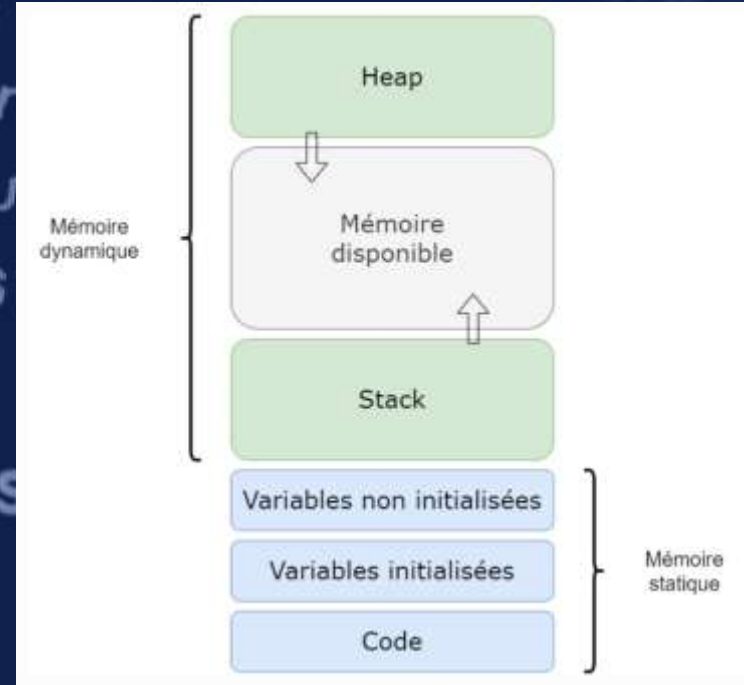
- le segment de code qui contient le code exécutable ;
- le segment de données qui contient les variables globales ;
- le segment de pile (**stack**) qui contient les variables locales et les adresses de retour des fonctions ;
- le segment de tas (**heap**) qui contient les variables allouées dynamiquement.



Mémoire - Organisation 2

La **mémoire statique** est utilisée pour stocker le code exécutable et les variables globales (variables déclarées en dehors de toute fonction).

La **mémoire de tas** (heap) est utilisée pour stocker les variables allouées dynamiquement (ce qui sera abordé dans le chapitre Allocation dynamique). La **mémoire de pile** (stack) est utilisée pour stocker les variables locales et les adresses de retour des fonctions.



Mémoire - La stack

Jusqu'à présent, la mémoire nécessaire à l'exécution des programmes a été allouée statiquement à la compilation (pour les variables globales), ou automatiquement durant l'exécution du programme, puis libérée à la fin de l'exécution du programme. La mémoire nécessaire était réservée dans une zone appelée la pile ou **stack**.

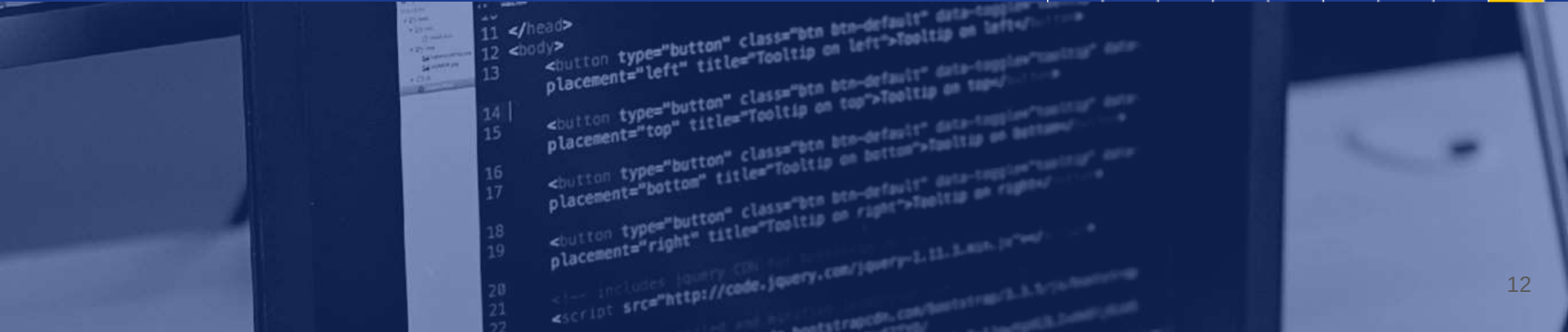
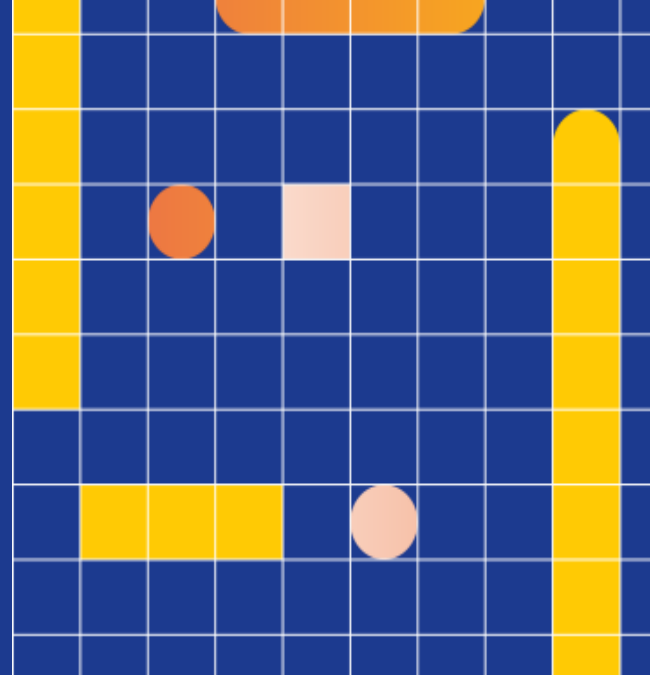
La stack est une zone mémoire intéressante pour les raisons suivantes :

- elle est gérée par le CPU ;
- la réservation de la mémoire est faite automatiquement ;
- la mémoire est automatiquement libérée lorsqu'on sort de la fonction.

Stack - Inconvénients

La stack a aussi des inconvénients :

- le compilateur a besoin de connaître l'encombrement mémoire avant l'exécution du programme. Ceci a généralement pour conséquence de surdimensionner la place réservée ;
- la taille est limitée par le système d'exploitation (généralement 8 Mb).



Mémoire - Le heap

Contrairement à la stack, le **heap** possède les caractéristiques suivantes:

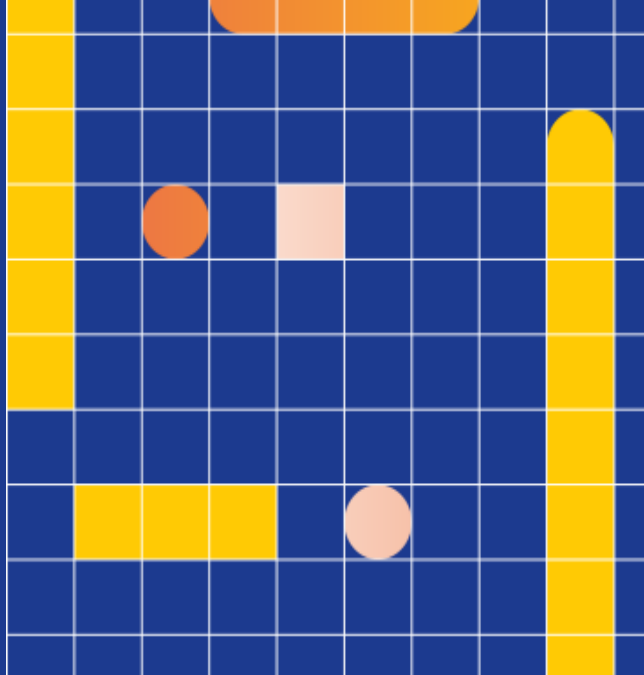
1. sa taille n'est pas fixée a priori. Elle n'est limitée que par la taille de la mémoire physique de la machine ;
2. l'allocation (et la désallocation) doit se faire manuellement ;
3. les variables doivent être manipulées avec des pointeurs.

Les langages modernes tels que Python gèrent automatiquement l'allocation et la désallocation mémoire, avec un dispositif appelé *garbage collector*. L'interpréteur contrôle cycliquement l'utilisation de la mémoire et libère les zones qui ne sont plus utilisées.

Heap - Gestion manuelle

La gestion manuelle de la mémoire est une des forces, mais également une des difficultés de la programmation en langage C :

- on évite le recours au *garbage collector* qui est (un peu) coûteux en temps de calcul ;
- mais il faut toujours penser à libérer la mémoire non utilisée sous peine de **fuites mémoire** (memory leak) qui affectent tout le système d'exploitation.



```
11 </head>
12 <body>
13   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="left" title="Tooltip on left">Tooltip on left</button>
14   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="top" title="Tooltip on top">Tooltip on top</button>
15   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="bottom" title="Tooltip on bottom">Tooltip on bottom</button>
16   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="right" title="Tooltip on right">Tooltip on right</button>
17
18   <!-- includes jquery CDN for tooltip -->
19   <script src="http://code.jquery.com/jquery-2.1.1.min.js"></script>
20   <script src="http://code.jquery.com/jquery-2.1.1.min.js"></script>
21   <script src="http://code.jquery.com/jquery-2.1.1.min.js"></script>
22   <script src="http://code.jquery.com/jquery-2.1.1.min.js"></script>
```



L'allocation dynamique de mémoire en C

Adapter le stockage au stock



ADM - Rôle

Il n'est pas toujours possible de savoir quelle quantité de mémoire sera utilisée par un programme.

Dans le cadre des tableaux, la limite imposée par le développeur peut gêner l'utilisateur. Et on utilise trop de mémoire si un tableau n'est pas utilisé entièrement.

Un ordinateur ne dispose que d'une quantité limitée de mémoire vive, il est donc important de ne pas en réserver abusivement. L'allocation dynamique permet de réserver une partie de la mémoire vive inutilisée pour stocker des données et de libérer cette même partie une fois qu'elle n'est plus nécessaire.

ADM - Objets et représentation

Tous les types du langage C manipulent des objets. En C un **objet** est une zone mémoire pouvant contenir des données et est composée d'un ou plusieurs multiplats contigus.

Chaque type utilise les objets de manière différente pour stocker les valeurs qui leur sont affectés. Ainsi, la valeur 1 n'est pas représentée de la même manière dans un objet de type **int** que dans un objet de type **double**.

Exemple de multiplats du stockage de la valeur 1 pour un **int** et pour un **float** :

1	1	0	0	0				
2	0	0	0	0	0	0	f0	3f

ADM - Rappel sur sizeof

L'opérateur `sizeof` permet de connaître la taille en multipliets (bytes en anglais) de son opérande. Cet opérande peut être soit un type (qui doit alors être entre parenthèses), soit une expression (auquel cas les parenthèses sont facultatives).

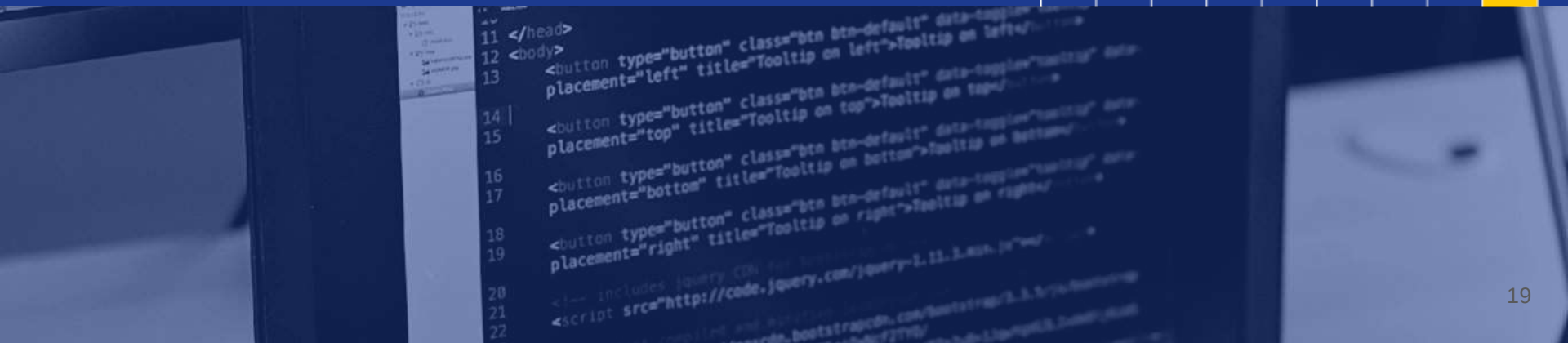
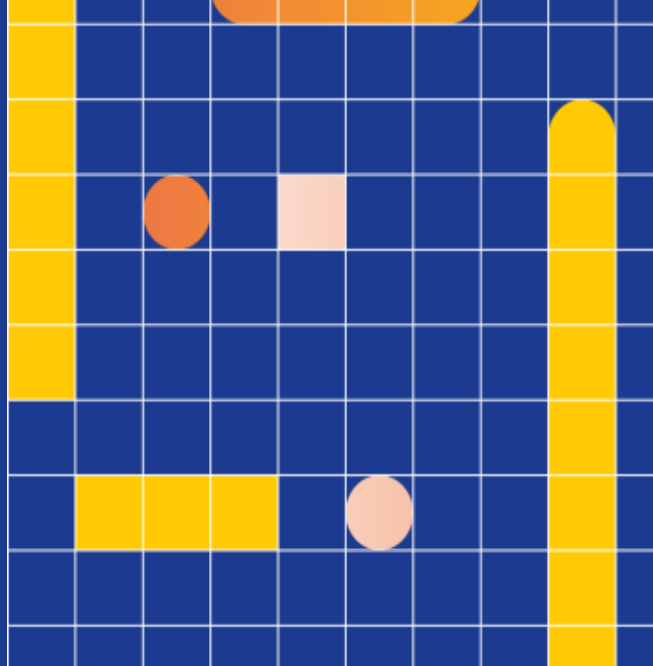
Le résultat de cet opérateur est de type `size_t`. Il s'agit d'un type entier non signé défini dans l'en-tête `<stddef.h>` qui est capable de contenir la taille de n'importe quel objet.

```
printf("_Bool : %zu\n", sizeof(_Bool));
```

ADM - Malloc

La fonction `malloc()` vous permet d'allouer un objet de la taille fournie en argument (qui représente un nombre de multipliants) et retourne l'adresse de cet objet sous la forme d'un pointeur générique. En cas d'échec de l'allocation, elle retourne un pointeur nul.

```
void *malloc(size_t taille);
```



ADM - Vérification du malloc

EXEMPLE

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    *p = 10;
16    printf("%d\n", *p);
17    return 0;
18 }
```

Dans l'exemple ci-contre, nous réservons un objet de la taille d'un `int`, nous y stockons ensuite la valeur dix et l'affichons. Pour cela, nous utilisons un pointeur sur `int` qui va se voir affecter l'adresse de l'objet ainsi alloué et qui va nous permettre de le manipuler comme nous le ferions s'il référençait une variable de type `int`.

ADM - malloc pour tableau

EXEMPLE

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int) * 10);
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    for (unsigned i = 0; i < 10; ++i)
16    {
17        p[i] = i * 10;
18        printf("p[%u] = %d\n", i, p[i]);
19    }
20
21    return 0;
22 }
```

Il est possible de simplifier l'expression fournie à `malloc()` en écrivant `sizeof(int[10])` qui a le mérite d'être plus concise et plus claire.

N'oubliez pas que `malloc()` n'effectue aucune initialisation.

ADM - Free

La fonction `free()` libère le bloc précédemment alloué par une fonction d'allocation dont l'adresse est fournie en argument. Dans le cas où un pointeur nul lui est fourni, elle n'effectue aucune opération.

```
void free(void *ptr);
```



ADM - Exemple free

EXEMPLE

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    *p = 10;
16    printf("%d\n", *p);
17    free(p);
18    return 0;
19 }
```

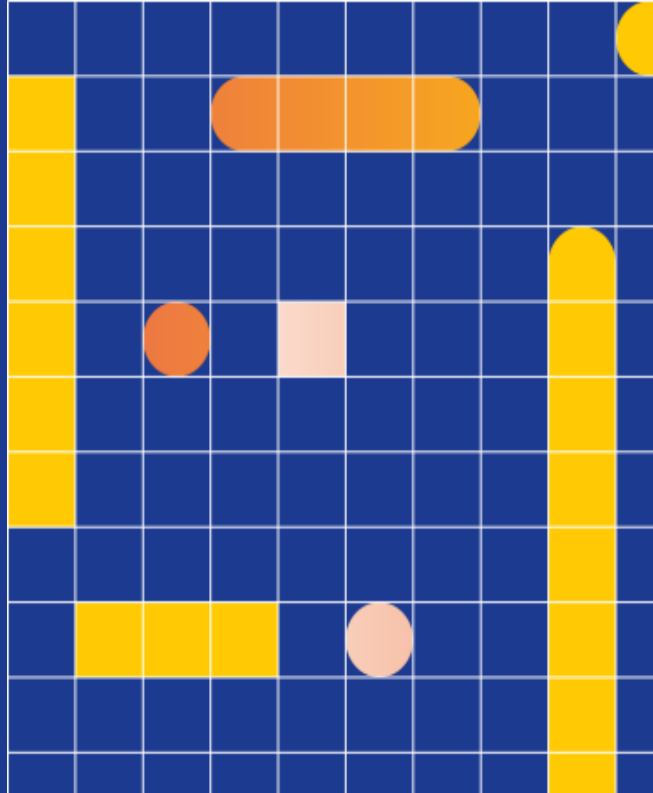
Dès que vous appelez une fonction d'allocation vous devez ensuite appeler la fonction `free()` après la manipulation afin de libérer la mémoire utilisée.

ADM - Calloc

La fonction `calloc()` attend deux arguments: le nombre d'éléments à allouer et la taille de chacun de ces éléments. Techniquement, elle revient au même que d'appeler `malloc()` comme suit: `malloc(nombre * taille);`

```
void *calloc(void *p, size_t taille);
```

à un détail près: chaque multiplet de l'objet ainsi alloué est initialisé à zéro.



```
11 </head>
12 <body>
13   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="left" title="Tooltip on left">Tooltip on left</button>
14   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="top" title="Tooltip on top">Tooltip on top</button>
15   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="bottom" title="Tooltip on bottom">Tooltip on bottom</button>
16   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="right" title="Tooltip on right">Tooltip on right</button>
17   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
    placement="left" title="Tooltip on left">Tooltip on left</button>
18   <button type="button" class="btn btn-default" data-toggle="tooltip" data-
```


ADM - Realloc

La fonction `realloc()` libère un bloc de mémoire précédemment alloué, en réserve un nouveau de la taille demandée et copie le contenu de l'ancien objet dans le nouveau. Dans le cas où la taille demandée est inférieure à celle du bloc d'origine, le contenu de celui-ci sera copié à hauteur de la nouvelle taille. À l'inverse, si la nouvelle taille est supérieure à l'ancienne, l'excédent n'est pas initialisé.

```
void *realloc(void *p, size_t taille);
```

La fonction attend deux arguments: l'adresse d'un bloc précédemment alloué à l'aide d'une fonction d'allocation et la taille du nouveau bloc à allouer. Elle retourne l'adresse du nouveau bloc ou un pointeur nul en cas d'erreur.

Realloc - Exemple

EXEMPLE

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      int *p = malloc(sizeof(int[10]));
8
9      if (p == NULL)
10     {
11         printf("Échec de l'allocation\n");
12         return EXIT_FAILURE;
13     }
14
15     for (unsigned i = 0; i < 10; ++i)
16         p[i] = i * 10;
17
18     int *tmp = realloc(p, sizeof(int[20]));
19
```

```
20     if (tmp == NULL)
21     {
22         free(p);
23         printf("Échec de l'allocation\n");
24         return EXIT_FAILURE;
25     }
26
27     p = tmp;
28
29     for (unsigned i = 10; i < 20; ++i)
30         p[i] = i * 10;
31
32     for (unsigned i = 0; i < 20; ++i)
33         printf("p[%u] = %d\n", i, p[i]);
34
35     free(p);
36     return 0;
37 }
```

Realloc - Fuite de mémoire

Remarquez que nous avons utilisé une autre variable, `tmp`, pour vérifier le retour de la fonction `realloc()`. En effet, si nous avions procédé comme ceci.

```
p = realloc(p, sizeof(int[20]));
```

Il nous aurait été impossible de libérer le bloc mémoire référencé par `p` en cas d'erreur puisque celui-ci serait devenu un pointeur nul. Il est donc impératif d'utiliser une seconde variable afin d'éviter des fuites de mémoire.

ADM - Tableaux de longueur variable

Les tableaux de longueur variable permettent aussi de demander au compilateur de s'adapter dynamiquement à la taille des tableaux que nous voulons manipuler.

```
1 int t1[3]; /* Taille déterminée */  
2 int t2[] = { 1, 2, 3 }; // Taille déterminable
```

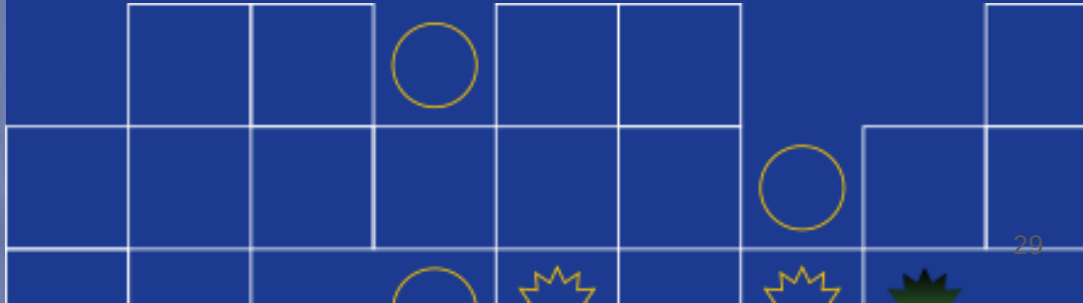
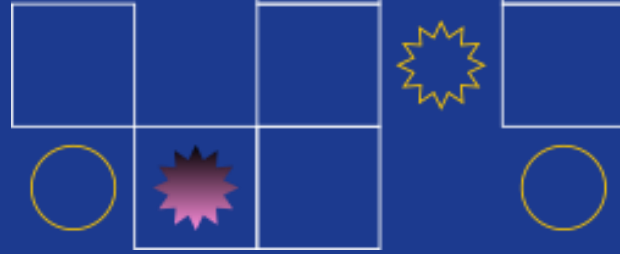
Toutefois cette méthode présente des limites :

- **durée de vie** - cette variable n'existe plus à la sortie d'une fonction ;
- la **taille** est plus limitée que pour les mallocs (négligeable) ;
- et la **gestion d'erreur** est plus compliquée.



La différence entre le compiletime et le runtime

Compilation et exécution



Temps de compilation et d'exécution

A cause de la spécificité des langages compilés, les erreurs peuvent arriver à deux moments différents :

- au moment de la **compilation**, on parle de *compiletime* ;
- ou au moment de l'**exécution** de l'exécutable, le *runtime*.

Les erreurs du compiletime et du runtime ont une nature différente et ne nécessitent pas les mêmes efforts pour être évitées ou résolues.

Erreurs du compiletime

Les erreurs qui arrivent lors de la compilation sont les plus simples : ce sont des erreurs de sémantique ou de syntaxe qui ont été repérées par le compilateur.

Elles ont été relevée avant l'édition de l'exécutable et le compilateur refusera de terminer sa tâche tant qu'elles n'auront pas été résolues.

Elles sont simples car le compilateur est capable de nous identifier ce qui cloche.

```
error: expected ';' before 'return'
```

Prévenir ces erreurs requiert de s'améliorer dans l'écriture du langage ou d'utiliser des outils de corrections pour gagner en efficacité.

Erreurs du runtime

Les erreurs qui arrivent lors du runtime sont plus problématiques pour le développeur.

Elles sont le résultat d'une erreur de calcul ou d'une fuite de mémoire et stoppent l'exécutable au plein milieu de son exécution.

```
Floating point exception (core dumped)
```

Cela implique plusieurs problèmes :

- il n'est pas possible de connaître l'existence de l'erreur **avant l'exécution** du programme qui implique des va et vient pour essayer de la résoudre.

Erreurs du runtime - Suite

Cela implique plusieurs problèmes :

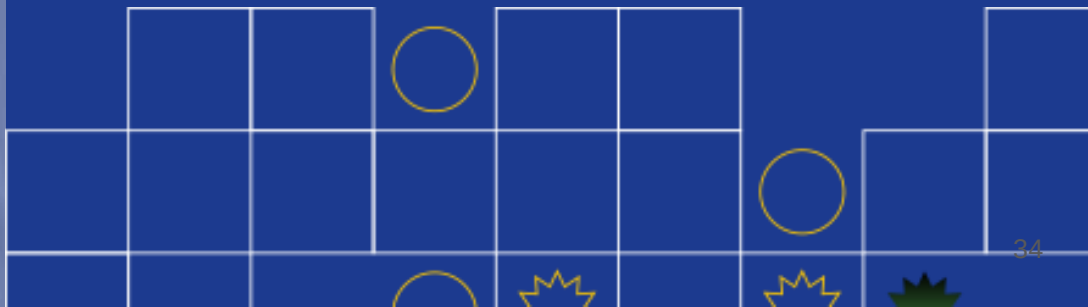
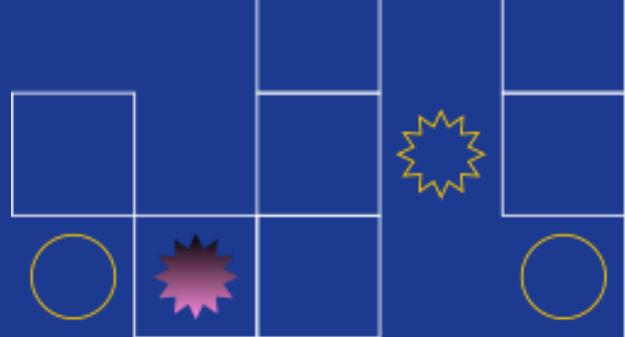
- Ces erreurs ne donnent **aucune information** sur les lignes qui ont pu créer ce problème calculé. Si le développeur est perdu, il a de fortes chances de revoir tout son code.
- Elles ont un lourd impact sur l'exécutable et la **qualité** du produit final, si elles ne sont pas anticipées correctement elles vont directement toucher l'utilisateur de l'exécutable.

Elles sont d'autant plus **impactantes** que les erreurs de compilations. Pour s'en prémunir il est important de prévoir des tests dans le code et de prévenir l'utilisateur (de l'exécutable) si quelque chose s'est mal passé.



Gestion des erreurs en C

Le choix entre la prévention et le chaos total.



GdE - Détection d'erreur

La première chose à faire pour gérer d'éventuelles erreurs lors de l'exécution, c'est avant tout de les détecter. Lorsque vous utilisez une fonction elle peut vous prévenir si elle s'est exécutée correctement ou non.



GdE - Scanf

EXEMPLE

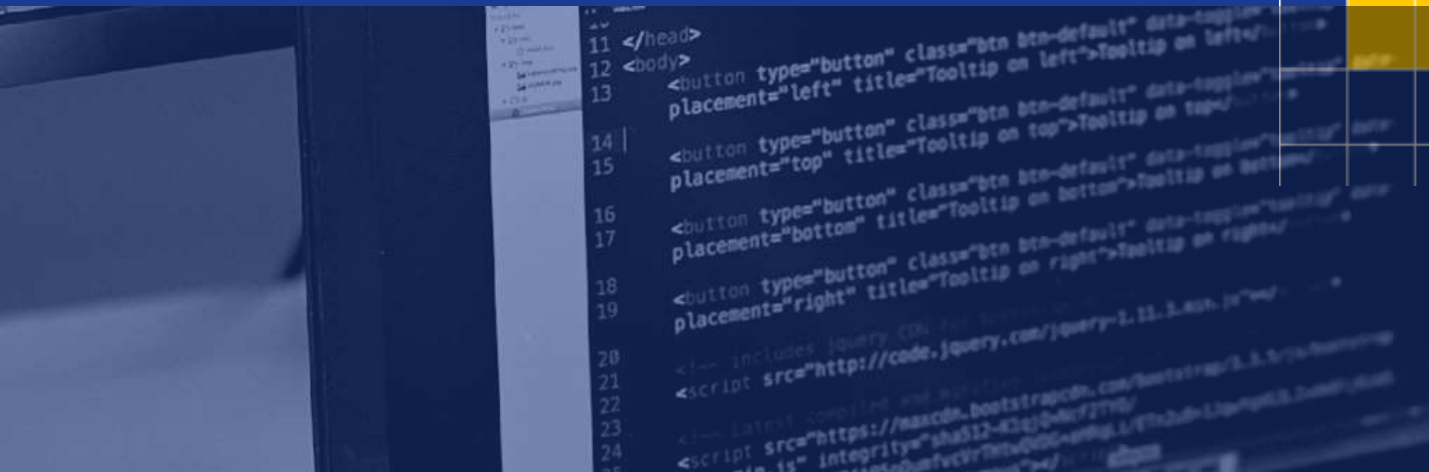
```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int x;
7      int y;
8
9      printf("Entrez deux nombres : ");
10
11     if (scanf("%d %d", &x, &y) == 2)
12         printf("Vous avez entre : %d et %d\n", x, y);
13
14     return 0;
15 }
```

La fonction `scanf()` retourne le nombre de conversions réussies ou un nombre inférieur si elles n'ont pas toutes été réalisées ou, enfin, un nombre négatif en cas d'erreur.

Cela permet aussi de vérifier que les données rentrées par l'utilisateur correspondent à ce qui a été demandé.

GdE - Fonctions

Quasiment toutes les fonctions renvoient des entiers pour indiquer si elles se sont déroulées correctement. Vous pouvez consulter [la norme](#) pour vérifier ce que renvoient les fonctions et solidifier votre code.



GdE - La variable globale Errno

Lorsque les fonction renvoient un résultat, il faut passer par un autre canal pour enregistrer une potentielle erreur.

La variable globale `errno` inclus dans l'entête `<errno.h>` permet de suivre les erreur de ces fonctions.

Si cette variable reste à la valeur 0 cela implique qu'aucune erreur n'est survenue dans l'exécution du programme. Elle doit être initialisée à 0 au début du code pour pouvoir fonctionner correctement.

GdE - Errno exemple

EXEMPLE

```
1  #include <errno.h>
2  #include <math.h>
3  #include <stdio.h>
4
5
6  int main(void)
7  {
8      errno = 0;
9      double x = pow(-1, 0.5);
10
11     if (errno == 0)
12         printf("x = %f\n", x);
13
14     return 0;
15 }
```

L'exemple ci-contre effectue un calcul qui sort du domaine des réels et du coup la valeur de `errno` sera modifiée pour notifier l'erreur.

GdE - Prévenir l'utilisateur

EXEMPLE

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int x;
7     int y;
8
9     printf("Entrez deux nombres : ");
10
11     if (scanf("%d %d", &x, &y) == 2)
12         printf("Vous avez entré : %d et %d\n", x, y);
13     else
14         printf("Vous devez saisir deux nombres !\n");
15
16     return 0;
17 }
```

Savoir qu'une erreur s'est produite, c'est bien, le signaler à l'utilisateur, c'est mieux. Ne laissez pas votre utilisateur dans le vide, s'il se passe quelque chose, dites le lui.

C'est comme ça que l'on fait de la prévention d'erreur de qualité !

GdE - Exit

EXEMPLE

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int x;
8     int y;
9
10    printf("Entrez deux nombres : ");
11
12    if (scanf("%d %d", &x, &y) != 2)
13    {
14        printf("Vous devez saisir deux nombres !\n");
15        return EXIT_FAILURE;
16    }
17
18    printf("Vous avez entré : %d et %d\n", x, y);
19    return 0;
20 }
```

Maintenant que vous savez tout cela, il vous est possible de modifier le code utilisant la fonction `scanf()` pour vérifier si celle-ci a réussi et, si ce n'est pas le cas,, préciser à l'utilisateur qu'une erreur est survenue et quitter la fonction `main()` en retournant la valeur `EXIT_FAILURE`.

ADM - Vérification du malloc

EXEMPLE

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    *p = 10;
16    printf("%d\n", *p);
17    return 0;
18 }
```

Lorsque vous attribuez de la mémoire vous devez toujours vérifier le retour d'une fonction d'allocation afin de vous assurer que vous manipulez bien un pointeur valide.

Vous pouvez utiliser le format ci-joint pour le faire et sortir du main avec une erreur pour éviter les fuites de mémoire.

GdE - Interpréter Errno

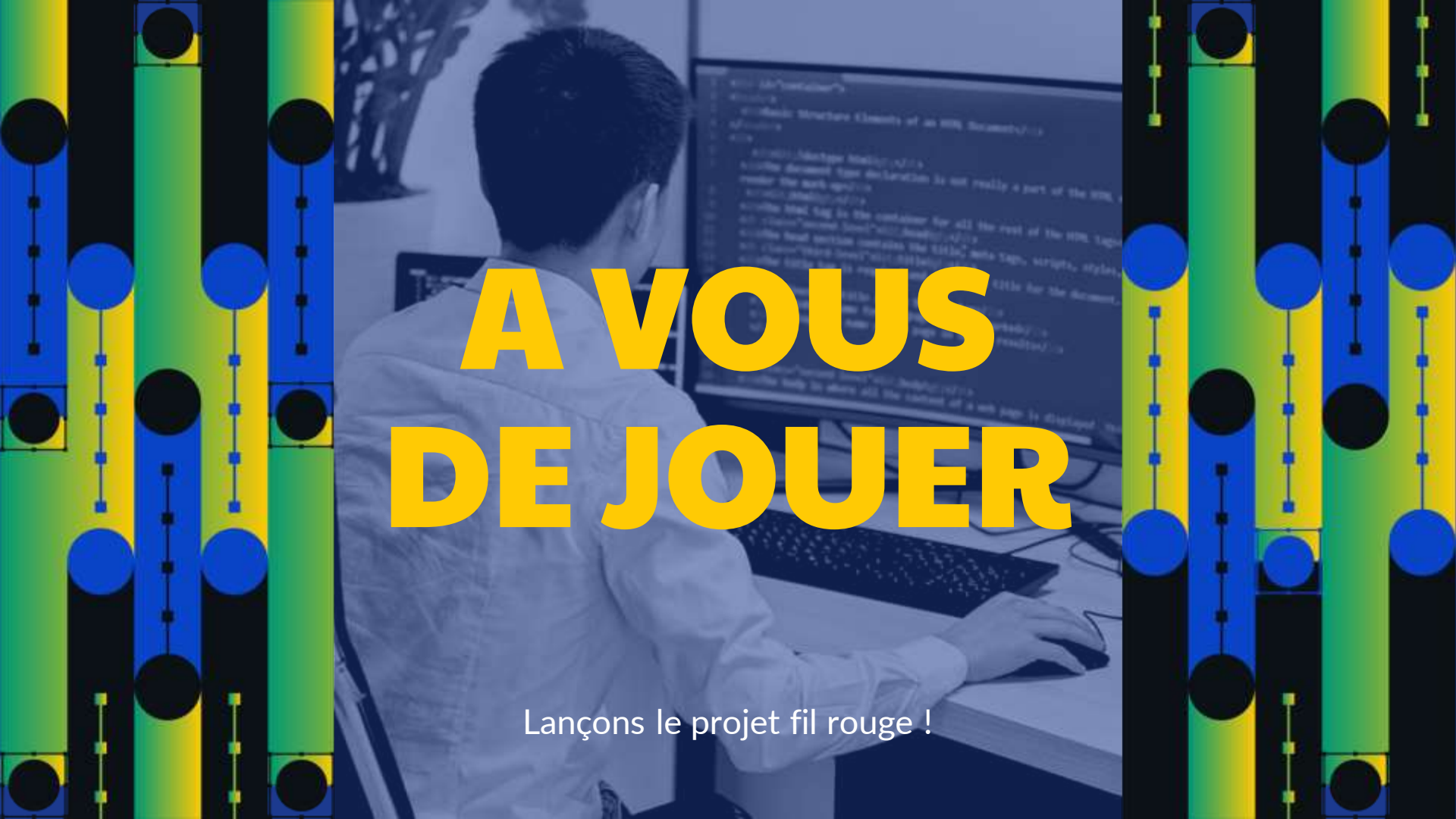
EXEMPLE

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6
7 int main(void)
8 {
9     FILE *fp;
10
11     fp = fopen("nawak.txt", "r");
12
13     if (fp == NULL)
14     {
15         fprintf(stderr, "fopen: %s\n", strerror(errno));
16         return EXIT_FAILURE;
17     }
18
19     fclose(fp);
20     return 0;
21 }
```

La fonction `strerror()` (déclarée dans l'en-tête `<string.h>`) retourne une chaîne de caractères correspondant à la valeur entière fournie en argument.

Cette fonction permet de traduire l'entier renvoyé par `errno` en un message d'erreur beaucoup plus clair !

```
fopen: No such file or directory
```



A VOUS DE JOUER

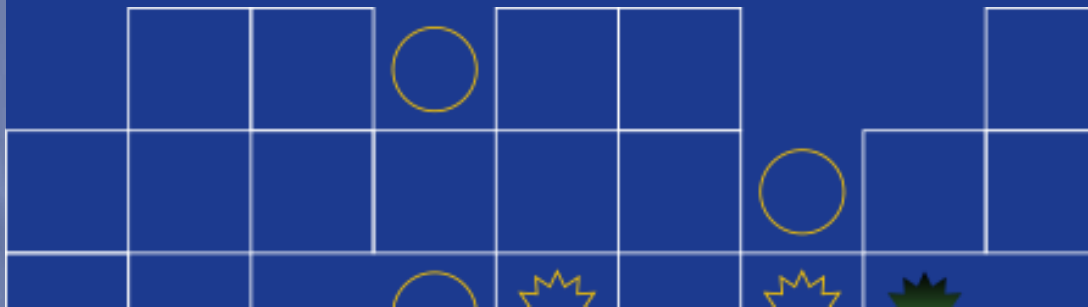
Lançons le projet fil rouge !



2048



Projet de jeu 2048 pour un projet console en C



2048 ▶ - Règles du jeu



Le jeu se déroule dans un carré de 16 cases ($4 * 4$). Deux de ces cases sont initialisées au début du jeu à la valeur 2.

Le joueur peut indiquer une direction à chaque tour : *haut* ; *bas* ; *gauche* ou *droite*. Tous les carrés se déplacent alors vers la première case disponible dans cette direction et s'empilent. Ensuite un 2 s'ajoute dans une case vide. Et c'est à nouveau au tour du joueur.

Si deux chiffres de même valeur s'empilent, ils fusionnent pour donner leur carré.

Le joueur gagne s'il arrive à atteindre la valeur 2048 et perd si tous ses mouvements sont bloqués.

2048 ▶ - Barème



	2		
	4	16	4
	2	8	4

Partie code /10

- Utilisation de la mémoire en C /4
- Structure et logique /3
- Clarté du code, indentation et commentaire /3

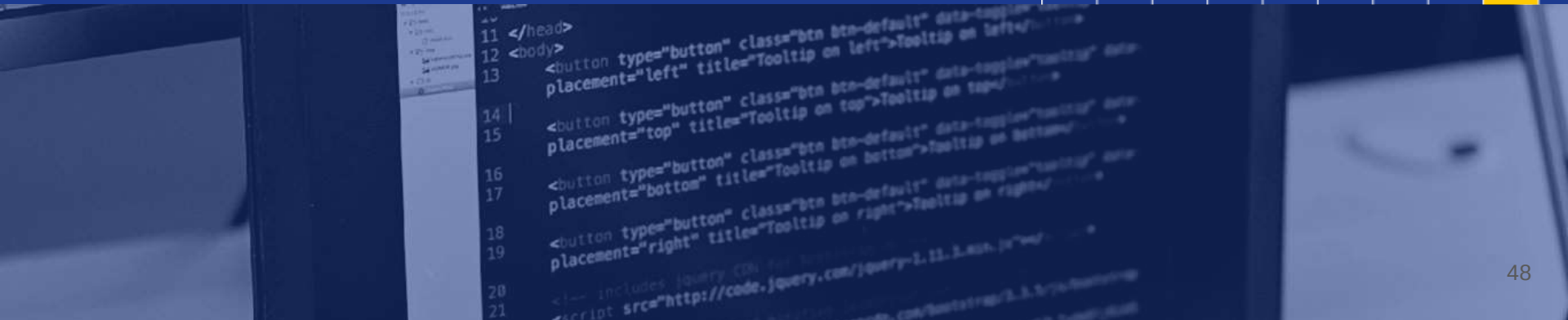
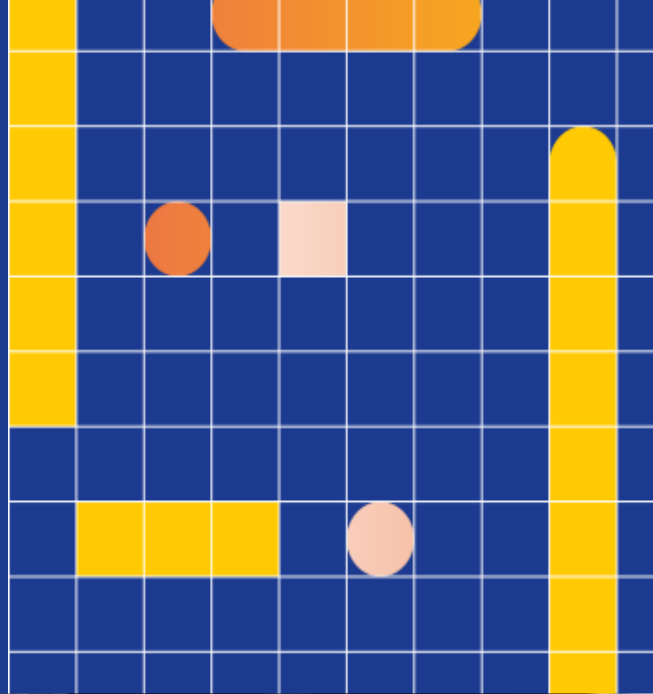
Partie présentation /10

- Réponse aux consignes /2
- Supports de présentation /2
- Aisance à l'oral et clarté /2
- Argumentation du projet /4

2048 ▶ - Présentation

Votre présentation de 5 minutes max lors du dernier cours doit inclure les pointes suivant :

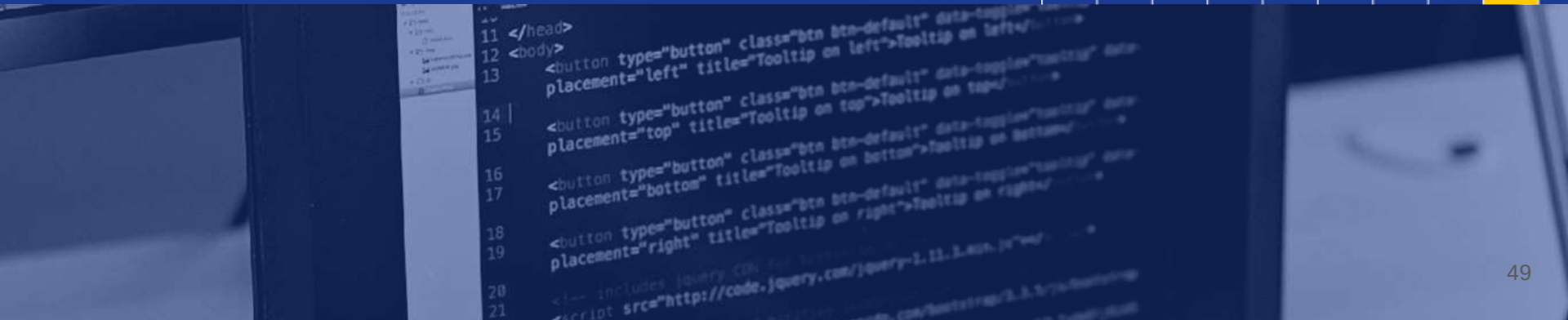
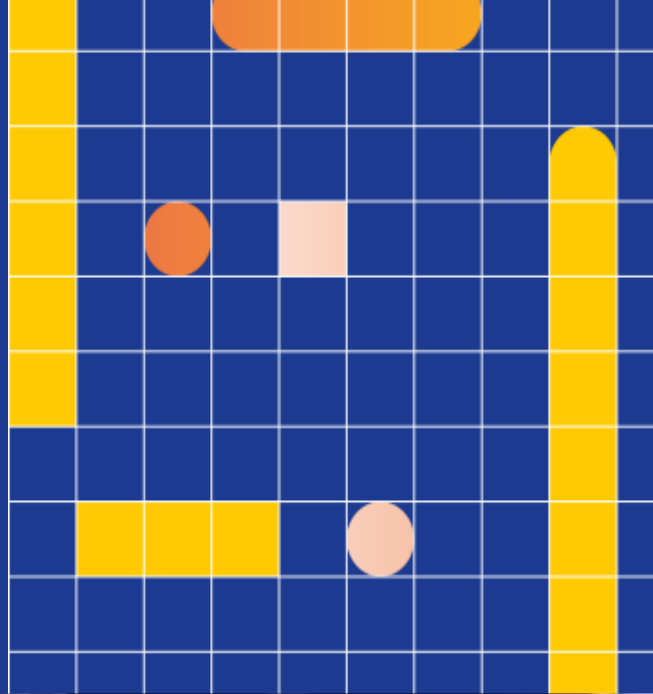
- une démonstration courte de la jouabilité de votre jeu ;
- Les difficulté que vous avez rencontré ;
- Les additions que vous avez eu le temps d'ajouter au projet (elles seront valorisées dans la notation) ;
- **La partie de code dont vous être le plus fier et pourquoi.**



2048 ▶ - Ajouts possibles

Voici des idées pour améliorer votre projet si cela rentre dans le cadre :

- améliorer l'affichage du jeu sur la console ;
- gérer le score ;
- permettre de recommencer ;
- changer la taille du terrain ;
- etc.



Liens utiles

Processus de compilation et programmer en C :

- <https://perso.esiee.fr/~courivad/C/rst/02-compilation.html>

Programmer en C :

- <https://openclassrooms.com/fr/courses/19980-apprenez-a-programmer-en-c>
- <https://zestedesavoir.com/tutoriels/755/le-langage-c-1/>